JETBRAINS | Kotlin

Collections and Co.

kotlinglang.org/education



What are they?

A collection usually contains a number of objects (this number may also be zero) of the same type.

Objects in a collection are called elements or items.

- Lists are ordered collections with access to elements by indices – integer numbers that reflect their position. Elements can occur more than once in a list.

- Sets are collections of unique elements. They reflect the mathematical abstraction of “set”: a group of objects without duplicates.

- Maps (or dictionaries) are sets of key-value pairs. The keys are unique, and each of them maps to exactly one value, while the values can be duplicated.

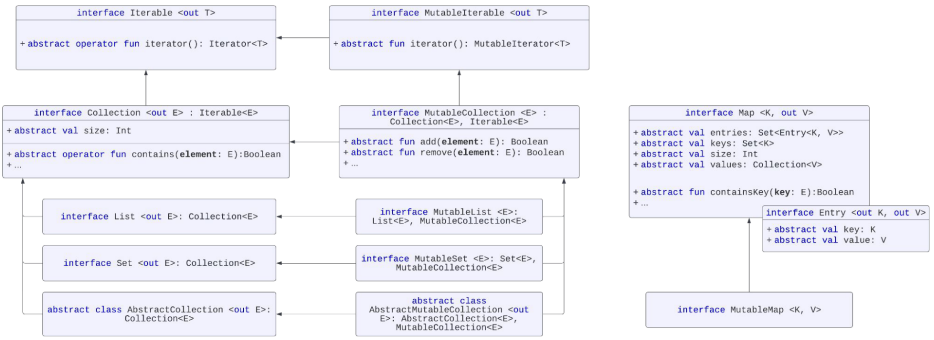
How can they be used?

Kotlin lets you manipulate collections independently of the exact types of objects stored in them.

In other words, you add a `String` to a list of `Strings` the same way as you would do with `Ints` or a user-defined class.

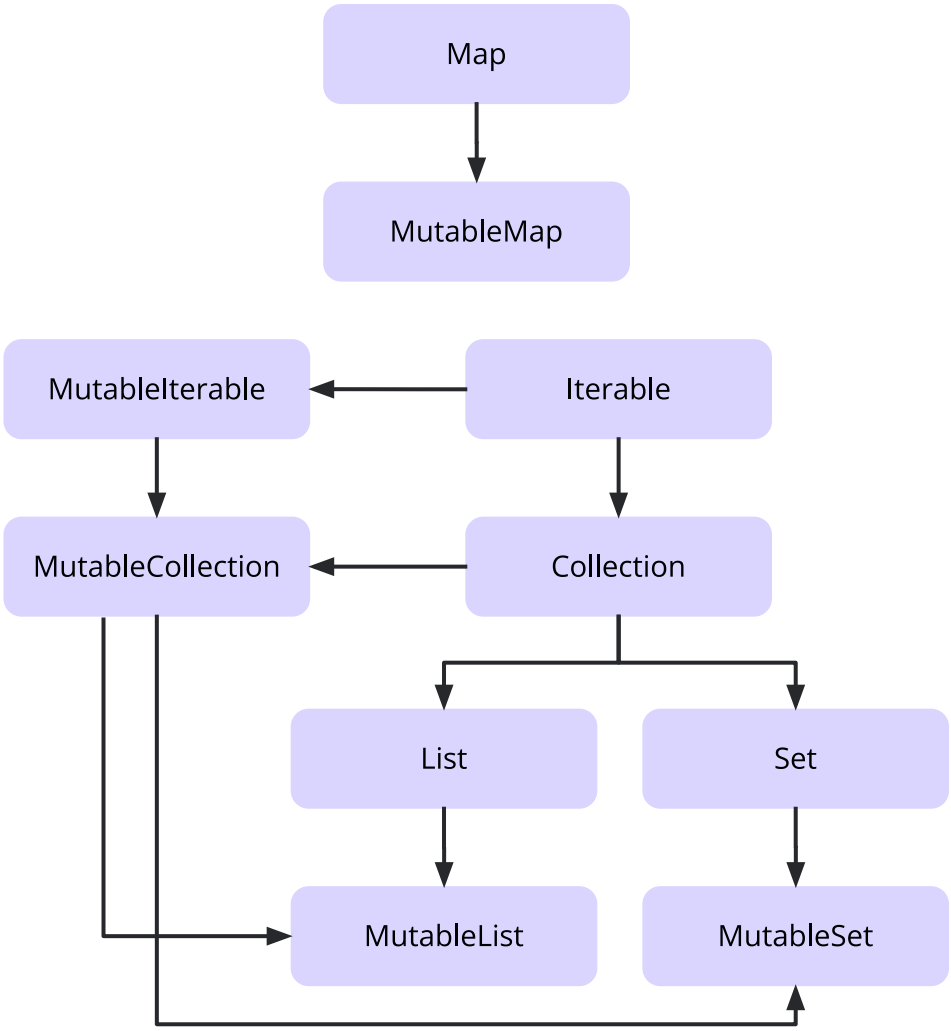
So, the Kotlin Standard Library offers generic interfaces, classes, and functions for creating, populating, and managing collections of any type.

Taxonomy of collections



Interfaces – Kotlin actually uses implementations from `java.util`

Taxonomy of collections



Iterable

All collections in Kotlin implement `Iterable` interface:

```
/**
* Classes that inherit from this interface can be
represented as a sequence of elements that can be
iterated over.
*
@param T is the type of element being iterated over.
The iterator is covariant in its element
type.
*/
public interface Iterable<out T> {
    // Returns an iterator over the elements of this
object.
   public operator fun iterator(): Iterator<T>
}
```

Iterable

All collections in Kotlin are `Iterable`:

```
val iterator = myIterableCollection.iterator()
while (iterator.hasNext()) {
   iterator.next()
}
```

Iterable vs MutableIterable

All collections in Kotlin are `Iterable`:

```
val iterator = myIterableCollection.iterator()
while (iterator.hasNext()) {
   iterator.next()
}
```

But some of them are `MutableIterable`:

```
val iterator =
myMutableIterableCollection.iterator()
while (iterator.hasNext()) {
   iterator.next()
   iterator.remove() // Because it is a mutable
iterator
}
```

Different kinds of collections

There are 2 kinds of collections: `Collection` and `MutableCollection`. `Collection` implements only `Iterable` interface, while `MutableCollection` implements `Collection` and `MutableIterable` interfaces.

`Collection` allows you to read values and make the collection **immutable**.

`MutableCollection` allows you to change the collection, for example by adding or removing elements. In other words, it makes the collection **mutable**.

```
val readonlyCollection = listOf(1, 2, 3)

readonlyCollection.add(4) // ERROR: Unresolved
reference: add

val mutableCollection = mutableListOf(1, 2, 3)

mutableCollection.add(4) // OK
```

Mutable Collection != Mutable Variable

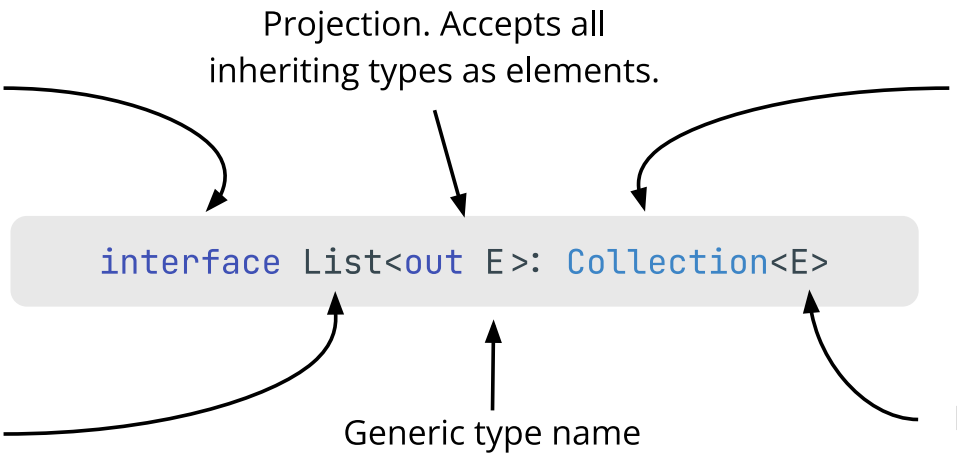
If you create a mutable collection, you **cannot** reassign the `val` variable.

```
val mutableCollection = mutableListOf(1, 2, 3)
mutableCollection.add(4) // OK
mutableCollection = mutableListOf(4, 5, 6) // ERROR: Val cannot be reassigned
```

But you can reassign var.

```
var mutableCollection = mutableListOf(1, 2, 3)
mutableCollection.add(4) // OK
mutableCollection = mutableListOf(4, 5, 6) // OK
```

The anatomy of a collection



AbstractionParent generic

Parent type name

Type name

The anatomy of a collection

Each collection has several **base** methods:

```
public interface Collection<out E> : Iterable<E> {
    public val size: Int

    public fun isEmpty(): Boolean

    public operator fun contains(element:
@UnsafeVariance E): Boolean

    public fun containsAll(elements:
Collection<@UnsafeVariance E>): Boolean
    ...
}
```



The anatomy of a collection

Actually there are **many** extensions:

Convenient to use in loops:

```
public val Collection<*>.indices: IntRange
    get() = 0..size - 1
for (i in collection.indices) { ... }

public val <T> List<T>.lastIndex: Int
    get() = this.size - 1

public inline fun <T> Collection<T>.isNotEmpty():
Boolean = !isEmpty()

...
```



Use this instead of `size ≠ 0`

Collections under the hood: List

```
public interface List<out E> : Collection<E> {
    public operator fun get(index: Int): E

    public fun indexOf(element: @UnsafeVariance E): Int
    public fun lastIndexOf(element: @UnsafeVariance E): Int

    public fun subList(fromIndex: Int, toIndex: Int): List<E>

    ...
}
```

Convenient to use with []: `collection[2]`

Make a **referenced** copy:

```
val list1 = mutableListOf(1, 2, 3)
val list2 = list1.subList(0, 1)
list1[0] += 1
println(list1) // [2, 2, 3]
println(list2) // [2]
```

Collections under the hood: List

To create a new list you can use special **builders** (by default `ArrayList`):

```
val list1 = emptyList<Int>() // Builds the internal
object EmptyList
val list2 = listOf<Int>() // Calls emptyList()
val list3 = listOf(1, 2, 3) // The type can be
inferred

val list4 = mutableListOf<Int>() // But better:
ArrayList<Int>()
val list5 = mutableListOf(1, 2, 3) // The type can be
inferred
val list6 = buildList {
        // constructs MutableList<Int>
        add(5)
        addAll(0, listOf(1, 2, 3))
    }
```

Collections under the hood: Set

```
public interface Set<out E> : Collection<E> {
     abstract val size: Int

abstract fun contains(element: @UnsafeVariance E):
Boolean

abstract fun containsAll(collection: Collection<E>):
Boolean

abstract fun isEmpty(): Boolean

}

abstract fun iterator(): Iterator<E>
```

A generic unordered collection of elements that does not support duplicate elements. It compares objects via the equals method instead of checking if the objects are the *same.*

Collections under the hood: Set

```
class A(val primary: Int, val secondary: Int)
class B(val primary: Int, val secondary: Int) {
    override fun hashCode(): Int = primary

    override fun equals(other: Any?) = primary ==
(other as? B)?.primary
}

fun main() {
    val a = A(1,1)
    val b = A(1,2)
    val set = setOf(a, b)
    println(set) // two elements
}
```

Collections under the hood: Set

```
class A(val primary: Int, val secondary: Int)
class B(val primary: Int, val secondary: Int) {
    override fun hashCode(): Int = primary

    override fun equals(other: Any?) = primary ==
(other as? B)?.primary
}

fun main() {
    val a = B(1,1)
    val b = B(1,2)
    val set = setOf(a, b)
    println(set) // only one element
}
```

Collections under the hood: Set

To create a new set you can use special **builders** (by default `LinkedHashSet`):

```
val set1 = emptySet<Int>() // Builds the internal
object EmptySet
val set2 = setOf<Int>() // Calls emptySet()
val set3 = setOf(1, 2, 3) // The type can be inferred

val set4 = mutableSetOf<Int>() // But better:
LinkedHashSet<Int>() or HashSet<Int>()
val set5 = mutableSetOf(1, 2, 3) // The type can be
inferred
val set6 = buildSet {
        // constructs MutableSet<Int>
        add(5)
        addAll(listOf(1, 2, 3))
    }
```

Collections under the hood: Map

```
public interface Map<K, out V> {
     public fun containsKey(key: K): Boolean

}

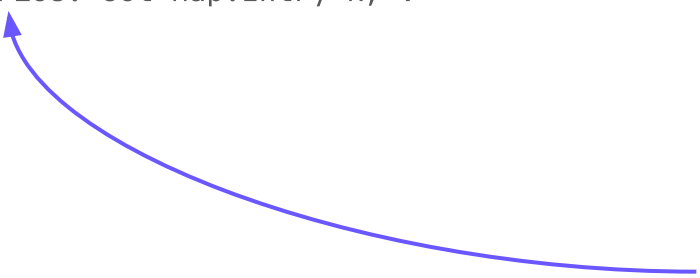
public fun containsValue(value: @UnsafeVariance V):
Boolean

public operator fun get(key: K): V?

public fun getOrDefault(key: K, defaultValue:
@UnsafeVariance V): V

 public val entries: Set<Map.Entry<K, V>>

 …
```



Convenient to use in loops:

```
for ((key, value) in map.entries) { ... }
```

Collections under the hood: Map

To create a new map you can use special **builders** (by default LinkedHashMap):

```
val map1 = emptyMap<Int, String>() // Builds the internal object EmptyMap
val map2 = mapOf<Int, String>() // Calls emptyMap()
val map3 = mapOf(1 to "one", 2 to "two") // The type can be inferred

val map4 = mutableMapOf<Int, String>() // But better: LinkedHashMap<...>() or HashMap<...>()
val map5 = mutableMapOf(1 to "one", 2 to "two") // The type can be inferred
val map6 = buildMap {
        // constructs MutableMap<Int, String>
        put(1, "one")
        putAll(mutableMapOf(2 to "two"))
    }
```

Array

- Not a collection and not iterable, but has an **iterator**.
- Has a **fixed** size, but its elements are **mutable**.

```
/**
 * Represents an array (specifically a Java array when
targeting the JVM platform).
 * Array instances can be created using the [arrayOf],
[arrayOfNulls], and [emptyArray] standard
library functions.
 */
public class Array<T> {
    public operator fun set(index: Int, value: T):
Unit

    …
}
```

Array

Kotlin also has classes that represent arrays of primitive types without boxing overhead: `ByteArray`, `ShortArray`, `IntArray`, and so on.

```
/**
 * An array of ints. When targeting the JVM, instances
of this class are represented as `int[]`.
 */
public class IntArray(size: Int) {
    public operator fun set(index: Int, value: T):
Unit

    …
}
```

Ranges

Not collections, but there are defined *progressions* for standard types with **iterators**: `CharProgression`, `IntProgression`, `LongProgression`:

```
for (c in 'a'..'c') { ... } // CharProgression
for (i in 1..5) { ... }     // IntProgression
for (i in 1L..5L) { ... }   // LongProgression
```

There are a lot of ways to customize them:

```
for (i in 10 downTo 0 step 3) { ... }
```

`downTo` and `step` infix extension functions.

Sequence

Not a collection, but has an **iterator**:

```
/**
 * A sequence that returns values through its
iterator. The values are evaluated lazily, and the
sequence is potentially infinite.
 */
public interface Sequence<out T> {
    public operator fun iterator(): Iterator<T>
}
```

Sequence

To create a new sequence you can use special **builders**:

```
val sequence1 = emptySequence<Int>() // Builds the
internal object EmptySequence
val sequence2 = sequenceOf<Int>() // Calls
emptySequence()
val sequence3 = sequenceOf(1, 2,
3) // The type can be inferred
val sequence4 = sequence {
     // constructs Sequence<Int>
     yield(1)
     yieldAll(listOf(2, 3))
}

val sequence5 = generateSequence(1) { it +
2 } // `it` is the previous element
println(sequence5.take(5).toList()) // [1, 3, 5, 7, 9]
```

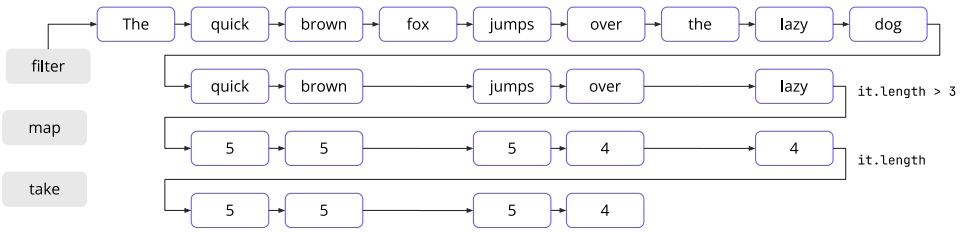
Sequence vs List

```
val words = "The quick brown fox jumps over the lazy
dog".split(" ") // Returns a list
val lengthsList = words.filter { println("filter:
$it"); it.length > 3 }
     .map { println("length: ${it.length}");
it.length }
     .take(4)

println("Lengths of first 4 words longer than 3
chars:")
println(lengthsList)
```

Sequence vs List

```
val words = "The quick brown fox jumps over the lazy
dog".split(" ") // Returns a list
```



Sequence vs List

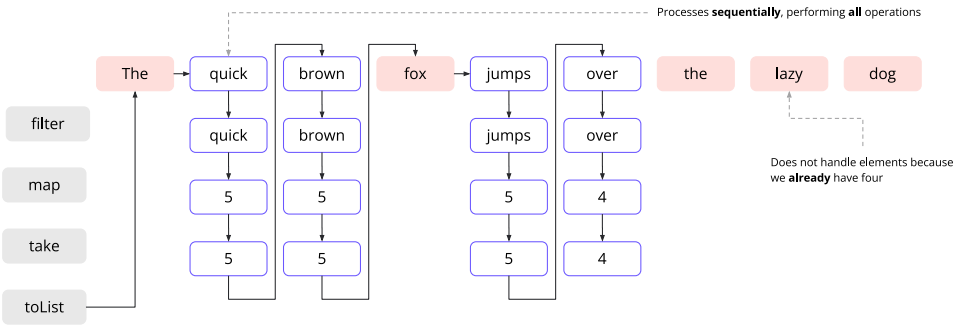
```
val words = "The quick brown fox jumps over the lazy
dog".split(" ") // Returns a list
// Convert the List to a Sequence
val wordsSequence = words.asSequence()
val lengthsSequence = wordsSequence.filter
{ println("filter: $it"); it.length > 3 }
     .map { println("length: ${it.length}");
it.length }
     .take(4)

println(lengthsSequence) // prints
`kotlin.sequences.TakeSequence@MEMORY_ADDR`

println("Lengths of first 4 words longer than 3
chars:")
// Terminal operation: obtaining the result as a List
println(lengthsSequence.toList()) // top code gets
executed, then prints `[5, 5, 5, 4]`
```

Sequence vs List

```
val words = "The quick brown fox jumps over the lazy
dog".split(" ") // Returns a list
val wordsSequence = words.asSequence()
```



Collection operations

There are **many** different functions for working with collections. If you need to do something with a collection, Google it first. Most likely, the standard library already has the function you need, for example:

```
public fun <T : Comparable<T>> List<T?
>.binarySearch(element: T?, fromIndex: Int = 0,
toIndex: Int
= size): Int

public actual fun <T : Comparable<T>>
MutableList<T>.sort(): Unit

public inline fun <T, K, V>
Iterable<T>.groupBy(keySelector: (T) → K,
valueTransform: (T) → V):
Map<K, List<V>>

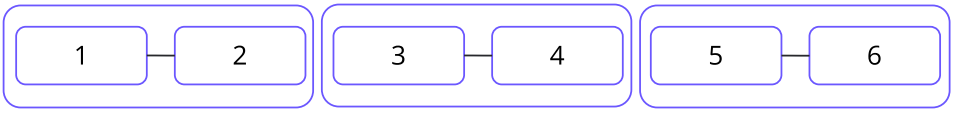
public inline
fun <T> Iterable<T>.partition(predicate: (T) →
Boolean): Pair<List<T>, List<T>>
```

Collection operations

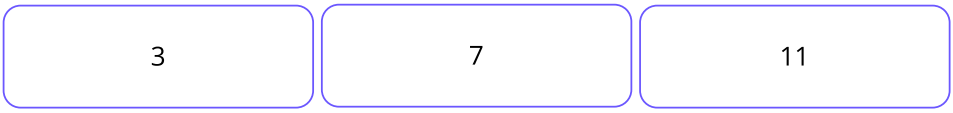



```
val exampleList = listOf(1, 2, 3, 4, 5, 6)
```

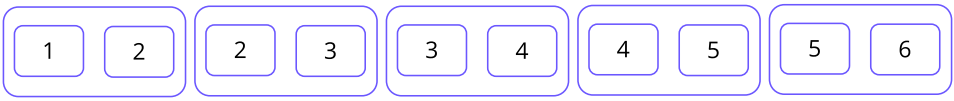
```
exampleList.chunked(2)
```




```
exampleList.chunked(2) { it.sum() }
```




```
exampleList.windowed(2)
```




```
exampleList.drop(1).intersect(List(6) { it - 1 })
```



We will look into even more operations in our future lecture on Functional programming.

JETBRAINS | Kotlin

Thank you

